

Memoria del Proyecto: Despliegue de Aplicación Flask en Alpine Linux

Proyecto: Fundamentos de Sistemas Operativos

Jorge Oís y Miguel Pajuelo

1. Introducción y Objetivos

En este proyecto hemos tenido como finalidad desplegar y automatizar un sistema completo sobre Alpine Linux. A partir de una máquina virtual mínima, se han desarrollado scripts en Bash y módulos en Python que permiten: instalar dependencias, levantar una aplicación Flask, procesar datos reales (Gowalla) para generar mapas HTML y ejecutar un benchmarking secuencial/paralelo de multiplicación de matrices, además de orquestar peticiones externas al servidor Flask desde el host.

Objetivos:

- Automatizar la configuración de Alpine Linux, la creación de usuarios y la instalación de paquetes mediante scripts Bash.
- Desplegar el proyecto Flask en un entorno virtual de Python, gestionando todas las dependencias desde requirements.txt.
- Procesar datos de Gowalla para generar estadísticas y mapas HTML por ciudad y por usuario, integrando scripts Bash y Python.
- Implementar y comparar la ejecución secuencial y paralela de la multiplicación de matrices, midiendo tiempos y *speedup*.
- Diseñar un script en Python (requests) capaz de lanzar peticiones POST/GET al servidor Flask con reintentos y descarga automática de resultados.

2. Instalación y Configuración del Entorno (Ejercicio 1)

2.1 Instalación de Alpine Linux en VirtualBox

Partimos de la máquina virtual de Alpine proporcionada en la práctica, instalada en VirtualBox con el disco principal en sda y modo sys. Tras arrancar como root, ejecutamos setup-alpine y seguimos el asistente: configuración del teclado en español, hostname alps, interfaz eth0 por DHCP y zona horaria UTC. Creamos el usuario normal alumnoimat con su contraseña y dejamos preparado el acceso SSH mediante openssh. Finalmente reiniciamos, desconectamos la ISO desde VirtualBox y ya trabajamos siempre con el disco instalado.

2.2 Instalación de Paquetes con Script Bash

Para el primer ejercicio escribimos el script `apartado1_JorgeOis_MiguelPajuelo.sh`, ejecutado como root. El script comprueba para cada paquete si está instalado con `apk list --installed | grep -q <paquete>` y, sólo si no aparece, lo instala con `apk add`. El fichero `requirements.txt` reúne todas las dependencias necesarias para ejecutar correctamente la aplicación Flask y los distintos servicios del proyecto. Entre ellas se incluyen librerías esenciales de análisis y cálculo como **numpy**, **pandas**, **scikit-learn**, **matplotlib** y **seaborn**, utilizadas para procesar datos, entrenar modelos y generar visualizaciones. También se incorporan herramientas para depuración y monitorización como **memory-profiler**, así como librerías orientadas a la generación de mapas y geodatos, como **geopy**, **folium** y **branca**, que se emplean en el ejercicio de análisis de movilidad. Finalmente, dependencias como **Flask**, **tqdm** e **imgkit** permiten levantar el servidor web, gestionar procesos largos y exportar contenido en distintos formatos.

2.3 Configuración de Permisos de Super Usuario

Una vez instalado `sudo`, habilitamos el repositorio community editando `/etc/apk/repositories` para descomentar su línea correspondiente y poder instalar todos los paquetes necesarios. Después añadimos `alumnoimat` al grupo de administración modificando `/etc/sudoers` con `visudo`, permitiendo que los usuarios del grupo `wheel` ejecuten comandos como superusuario. Finalmente incluimos `alumnoimat` en `wheel`, de forma que a partir de ese momento todo el trabajo de despliegue y pruebas se realiza con este usuario usando `sudo` en lugar de `root`.

```
alps:~# echo '%wheel ALL=(ALL) ALL' > /etc/sudoers.d/wheel
alps:~# adduser alumnoimat wheel
alps:~# sudo -IU alumnoimat
Matching Defaults entries for alumnoimat on alps:
    secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

Runas and Command-specific defaults for alumnoimat:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR VISUAL"

User alumnoimat may run the following commands on alps:
    (ALL) ALL
```

Imagen 1. Configuración del grupo `wheel` y verificación de permisos `sudo` para el usuario `alumnoimat`.

3. Despliegue del Proyecto Flask (Ejercicio 2)

Para automatizar el despliegue del proyecto desarrollamos el script `apartado_despliegue_Bash_JorgeOis_MiguelPajuelo.sh`, ejecutado como `alumnoimat`. El script realiza un git clone del repositorio ProyectoFUSO, crea un entorno virtual de Python con `python3 -m venv ProyectoFUSO/venv`, lo activa y ejecuta `pip install -r requirements.txt` con todas las dependencias previamente recopiladas. Por último lanza la aplicación mediante `python3 ProyectoFUSO/main.py`, dejando el servidor Flask accesible en `http://<ip_alpine>:5000/`, donde se pueden probar los servicios de entrenamiento, estadísticas, EDA, generación de datasets sintéticos y benchmarking paralelo.

```

* Serving Flask app "main"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.16.10.90:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 138-640-614
172.16.12.38 - - [30/Nov/2025 09:59:32] "GET / HTTP/1.1" 200 -
172.16.12.38 - - [30/Nov/2025 10:00:58] "GET /compare_execution HTTP/1.1" 200 -

```

Imagen 2. Ejecución del servidor Flask en Alpine y registro de peticiones entrantes.

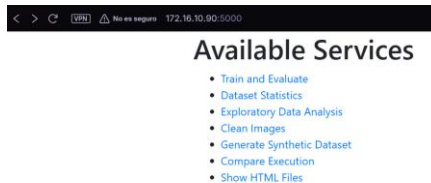


Imagen 3. Página principal del proyecto Flask mostrando los servicios disponibles.

3.1 Implementación de Comparación de Ejecución

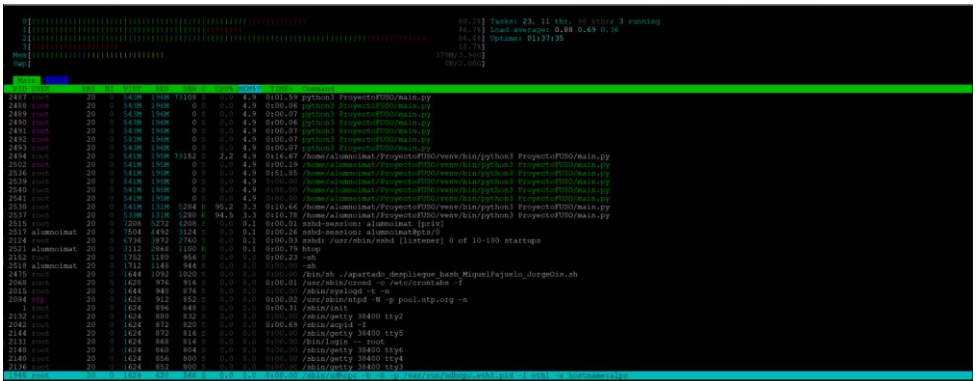
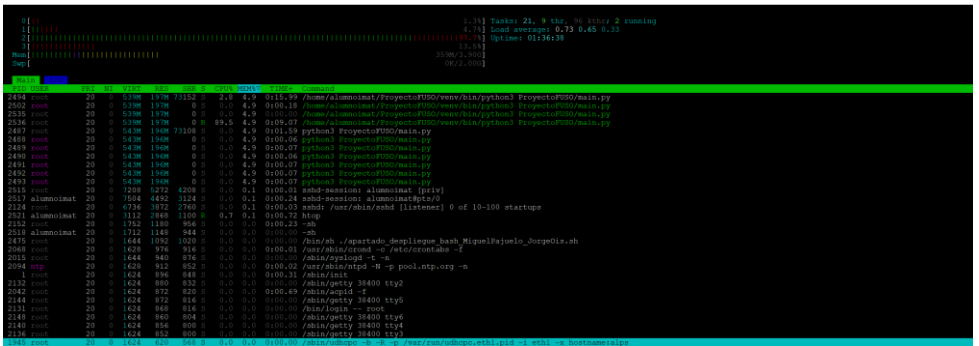
En el servicio Compare Execution implementamos la función `compare_execution()` que calcula el tiempo de ejecución secuencial y paralelo de la multiplicación de matrices para varios tamaños. La función genera matrices sintéticas, llama a la versión secuencial y a la versión paralela (basada en varios procesos o hilos) y devuelve una tupla con ambos tiempos medios. Al invocarlo desde la interfaz web y monitorizar con htop, se observa cómo la versión secuencial sólo utiliza un core mientras que la versión paralela reparte la carga entre varios, aumentando el uso de CPU a costa de un mayor consumo de memoria, lo que permite comparar claramente ambas estrategias.

Execution Comparison Results

Sequential Execution Time: 50.74238586425781 seconds

Parallel Execution Time: 33.274109840393066 seconds

Imagen 4. Resultados del servicio *Compare Execution* mostrando los tiempos de ejecución secuencial y paralela.



4. Procesamiento de Datos y Mapas (Ejercicio 3)

Para el procesamiento de datos de Gowalla desarrollamos el script `apartado_datos_mapas_JorgeOis_MiguelPajuelo.sh`. Este script descarga y descomprime los ficheros con `wget` y `unzip`, y genera ficheros intermedios como `ALL_LOCATIONS.txt` y sus versiones filtradas usando `cut`, `grep`, `sort -u` y contadores como `wc -l`. A partir de estos datos, los módulos `stats_checker_JorgeOis_MiguelPajuelo.py` y `topn_selection_JorgeOis_MiguelPajuelo.py` replican en Python las estadísticas obtenidas en terminal y extraen el top de usuarios más activos. Finalmente, `generate_maps.py` y `generate_individual_maps.py` crean mapas HTML para cada ciudad y para los usuarios seleccionados, que se pueden visualizar desde el navegador.



Imagen 7. Mapa generado a partir de los datos de Gowalla mostrando los desplazamientos de los usuarios en El Paso.

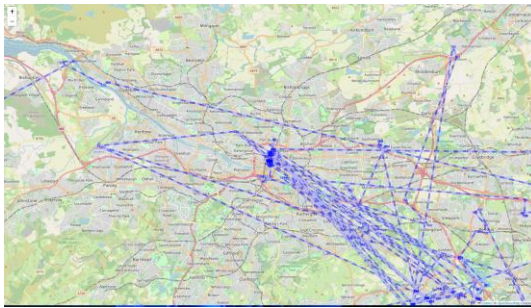


Imagen 8. Visualización de rutas y check-ins geolocalizados mediante Folium para Glasgow.

5. Automatización de peticiones con request (Ejercicio 4)

En el último ejercicio implementamos el script `peticiones_requests_JorgeOis_MiguelPajuelo.py`, que automatiza las llamadas al endpoint `/train` del servidor Flask. El script construye peticiones POST a `/train` con distintos pares de `train_size` y `test_size`, y tras cada entrenamiento descarga la figura generada mediante peticiones GET a la ruta `/static/` correspondiente. Para hacer el proceso robusto se diseñó una función `fetch_with_retry` que reintenta la petición varias veces ante errores de red o códigos de estado distintos de 200. De esta forma se genera de manera automática un conjunto completo de resultados y gráficas para analizar el rendimiento del modelo RandomForest.

```
Se van a probar 9 para iris/RandomForest.
--- Procesando Train: 0.1 / Test: 0.9 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.1Tst0.9.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.1Tst0.9.png

--- Procesando Train: 0.2 / Test: 0.8 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.2Tst0.8.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.2Tst0.8.png

--- Procesando Train: 0.3 / Test: 0.7 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.3Tst0.7.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.3Tst0.7.png

--- Procesando Train: 0.4 / Test: 0.6 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.4Tst0.6.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.4Tst0.6.png

--- Procesando Train: 0.5 / Test: 0.5 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.5Tst0.5.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.5Tst0.5.png

--- Procesando Train: 0.6 / Test: 0.4 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.6Tst0.4.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.6Tst0.4.png

--- Procesando Train: 0.7 / Test: 0.3 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.7Tst0.3.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.7Tst0.3.png

--- Procesando Train: 0.8 / Test: 0.2 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.8Tst0.2.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.8Tst0.2.png

--- Procesando Train: 0.9 / Test: 0.1 ---
Petición POST exitosa.
Descargando imagen desde: http://10.120.159.27:5000/static/irisTr0.9Tst0.1.png
EXITO: Imagen guardada en descargas_resultados_flask\irisTr0.9Tst0.1.png

--- PROCESO FINALIZADO ---
```

Imagen 9. Ejecución del script de peticiones automáticas mostrando el envío de solicitudes POST al servidor Flask y la descarga de las imágenes generadas para cada combinación de entrenamiento y test.

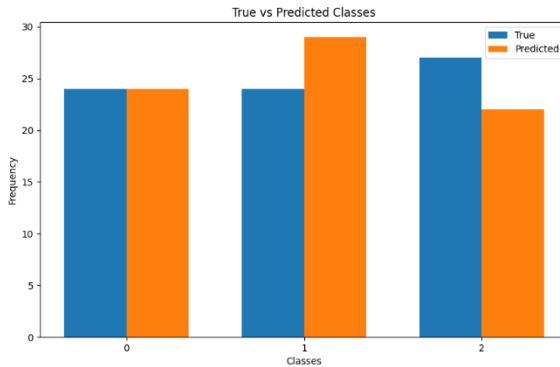


Imagen 10. Comparación gráfica entre las clases reales y las clases predichas tras el entrenamiento del modelo.

6. Conclusiones y Desafíos

Durante el proyecto los principales desafíos estuvieron relacionados con la gestión de dependencias en Alpine (necesidad de python3-dev, g++ y cabeceras para compilar algunas librerías), la configuración correcta de sudo para trabajar con un usuario no privilegiado y la integración de nuestro código de benchmarking y mapas dentro de la estructura ya existente del proyecto Flask.

Como posibles mejoras, sería interesante aislar aún más la lógica de negocio en módulos independientes, añadir tests automáticos para los scripts de Bash y los servicios Flask y documentar mejor los parámetros de cada endpoint, lo que facilitaría extender el proyecto a nuevos datasets o modelos en el futuro.